

Error Handling in JavaScript

Errors are inevitable in any application. JavaScript provides robust tools to **detect**, **handle**, and **respond** to runtime errors, allowing your code to fail gracefully.

Types of Errors

- **Syntax Errors:** Mistakes in the code structure (e.g., missing brackets)
 - **Runtime Errors:** Occur during execution (e.g., accessing undefined variables)
 - **Logical Errors:** Code runs but doesn't behave as intended
-

The `try...catch` Statement

Use `try...catch` to handle errors without crashing the entire script.

Syntax:

```
try {  
    // Code that may throw an error  
} catch (error) {  
    // Handle the error  
}
```

Example:

```
try {  
    let result = 10 / x; // x is not defined  
} catch (error) {  
    console.log('An error occurred:', error.message);  
}
```

The `finally` Block

The `finally` block is optional and always runs, **whether an error occurred or not**.

```
try {  
    // Risky code  
} catch (error) {  
    // Handle error  
} finally {  
    // Always runs  
    console.log('Cleanup complete');  
}
```

Throwing Custom Errors

You can manually throw errors using the `throw` statement.

```
function divide(a, b) {  
    if (b === 0) {  
        throw new Error('Cannot divide by zero');  
    }  
    return a / b;  
}  
  
try {  
    divide(5, 0);  
} catch (error) {  
    console.error(error.message); // Output: Cannot divide by zero  
}
```

Catching Specific Error Types

The `catch` block receives an `Error` object with useful properties:

```
try {  
  JSON.parse('invalid JSON');  
} catch (error) {  
  console.log(error.name);    // SyntaxError  
  console.log(error.message); // Unexpected token i in JSON  
}
```

Best Practices

- Always handle expected errors (e.g., API failures, user input)
 - Avoid swallowing errors silently — log them
 - Use specific error messages for debugging
 - Don't use try-catch for control flow in performance-critical code
-

Summary

- Use `try...catch` to prevent your app from crashing.
- Add `finally` for cleanup logic.
- Use `throw` to raise custom errors.
- Proper error handling makes applications more stable and user-friendly.